Ternary, Selections (`switch`), and Loops

Computer Programming 1 · Lecture 4

Siyavushkhon Kholmatov — New Uzbekistan University

Week 4 · Fall 2026

The conditional "Ternary" operator `?:`

Four lines, or one

With `if-else`

```
int larger;
if (a > b) {
    larger = a;
} else {
    larger = b;
}
```

With the conditional operator

```
int larger = (a > b) ? a : b;
```

Same logic, same result — but now the decision *is* the value being stored.

Read it as a question

```
condition ?  value if true :  value if false
```

It is the only C++ operator that takes three operands — hence its nickname, the *ternary* operator.

It is an expression, so it goes anywhere a value goes

```
int n = 7;

cout << n << " is " << (n % 2 == 0 ? "even" : "odd") << endl;   // inside a cout
int fee          = isStudent ? 5000 : 12000;                    // in an initialiser
double absValue  = (x < 0) ? -x : x;                            // in a calculation
```

Output of the second line

```
7 is odd
```

One expression needs one type

```
cout << (true ? 1 : 2.5) << endl;

cout << (false ? 1 : 2.5) << endl;

// cout << (true ? 1 : '2');  // ERROR
```

One expression needs one type

```
cout << (true ? 1 : 2.5) << endl;

cout << (false ? 1 : 2.5) << endl;

// cout << (true ? 1 : '2');  // ERROR
```

Output

```
1
2.5
```

Both branches are converted to one common type — before anything runs

`1` and `2.5` become `double`, so the first line produces `1.0` and prints `1`. An `int` and a string have no common type, so that third line is a compile error.

The condition itself is a `bool`, exactly like the one in an `if`.

Nesting — and when to stop

```
char grade = (score >= 90) ? 'A'
           : (score >= 75) ? 'B'
           : (score >= 60) ? 'C'
                           : 'F';
```

Works like an `else if` chain

Tested top to bottom; the first true one wins. Order still matters.

Use it for a value, not for actions

If a branch has to *do* something — print, read input, change two variables — use `if-else`.

Your turn — predict the output

```cpp
int a = 7, b = 7, x = 5;

cout << (a > b ? "a" : a < b ? "b" : "equal"
    )
     << endl;

cout << (x % 2 ? "odd" : "even") << " "
     << (x > 0 ? 1 : -1) * x << endl;
```

Your turn — predict the output

```cpp
int a = 7, b = 7, x = 5;

cout << (a > b ? "a" : a < b ? "b" : "equal"
    )
     << endl;

cout << (x % 2 ? "odd" : "even") << " "
     << (x > 0 ? 1 : -1) * x << endl;
```

Output

```
equal
odd 5
```

`x % 2` is `1`, and any non-zero value counts as `true`.

`switch` and `case`

When every test compares the same variable

An `else if` chain

```
if (choice == 1)
    cout << "Deposit";
else if (choice == 2)
    cout << "Withdraw";
else if (choice == 3)
    cout << "Balance";
else
    cout << "Unknown";
```

The same thing as a `switch`

```
switch (choice) {
    case 1:
        cout << "Deposit";
        break;
    case 2:
        cout << "Withdraw";
        break;
    case 3:
        cout << "Balance";
        break;
    default:
        cout << "Unknown";
}
```

The variable is written once. All branches test the same thing.

What `switch` can and cannot test

Allowed

- `int`, `char`, `bool` — any integral value
- `case` labels that are constants: `case 3:`, or `case 'A':`
- Each label distinct

Not allowed

- `double` or `string` in the `switch`
- Ranges — `case 1..5:` does not exist
- Conditions — `case x > 5:` is not a constant
- A variable as a label

So `switch` answers "*which one of these exact values?*". Everything else stays an `else if` chain.

`switch` jumps in — but never jumps out

```
int day = 3;

switch (day) {
    case 1: cout << "Mon ";
    case 2: cout << "Tue ";
    case 3: cout << "Wed ";
    case 4: cout << "Thu ";
    case 5: cout << "Fri ";
}
```

`switch` jumps in — but never jumps out

```
int day = 3;

switch (day) {
    case 1: cout << "Mon ";
    case 2: cout << "Tue ";
    case 3: cout << "Wed ";
    case 4: cout << "Thu ";
    case 5: cout << "Fri ";
}
```

Output

```
Wed Thu Fri
```

Execution enters at `case 3` and then keeps running straight through the labels below it.

A case label is a door in, not a box

Only `break` (or the closing brace) stops it. A forgotten `break` is the single most common `switch` bug — and it compiles without a word.

Fall-through on purpose: grouping cases

```
int month = 4, days;

switch (month) {
    case 1: case 3: case 5: case 7:
    case 8: case 10: case 12:
        days = 31;  break;
    case 4: case 6: case 9: case 11:
        days = 30;  break;
    case 2:
        days = 28;  break;
    default:
        days = 0;
}
cout << days << endl;    // 30
```

Why this reads well

Several labels share one body: "any of these values → do this".

`default` catches everything else

Put it last, and always have one — it is where invalid input goes instead of silently doing nothing.

Worked example — a tiny calculator

```
double a, b;  char op;
cin >> a >> op >> b;

switch (op) {
    case '+': cout << a + b;  break;
    case '-': cout << a - b;  break;
    case '*': cout << a * b;  break;
    case '/':
        if (b == 0) cout << "Cannot divide by zero";

        else        cout << a / b;
        break;
    default:  cout << "Unknown operator";
}
```

Input	Output
`8 * 5`	`40`
`7 / 2`	`3.5`
`7 / 0`	`Cannot divide...`
`7 ^ 2`	`Unknown operator`

The operator is a `char`, so `switch` is the natural fit. Note `a / b` is `double` division here.

Two `switch` traps

Declaring inside a case

```
switch (n) {
    case 1:
        int t = 5;    // ERROR
        cout << t;
        break;
    case 2:
        cout << "two";
}
```

All cases share one scope. Wrap the body in `{ }` and it compiles.

`break` in the wrong place

```
switch (grade) {
    case 'A':
    case 'B':
        cout << "Pass";
    case 'F':
        cout << "Fail";
        break;
}
```

`'A'` prints `PassFail`. The missing `break` is one line above `case 'F'`.

Your turn — predict the output

```
int n = 2;
switch (n) {
    case 1:
        cout << "one ";
    case 2:
        cout << "two ";
    case 3:
        cout << "three ";
        break;
    case 4:
        cout << "four ";
    default:
        cout << "other ";
}
cout << "done";
```

Your turn — predict the output

```
int n = 2;
switch (n) {
    case 1:
        cout << "one ";
    case 2:
        cout << "two ";
    case 3:
        cout << "three ";
        break;
    case 4:
        cout << "four ";
    default:
        cout << "other ";
}
cout << "done";
```

Output

```
two three done
```

Entry at `case 2`; `case 3` runs too; its `break` leaves the `switch`, so `four` and `other` never run.

`switch` or `else if`?

	`switch`	`else if` chain
Tests	one integral value against constants	any condition at all
Ranges, `&&`, `||`	× not possible	✓ natural
`double`, `string`	×	✓
Many equal values	✓ shortest and clearest	long and repetitive
Risk	a forgotten `break`	a mis-ordered condition

Menus, operator characters, month numbers, grade letters → `switch`.

Ranges, limits, combined conditions → `else if`.

Iteration: `while`

Every loop has the same three parts

From Week 2, in pseudocode

```
SET i = 1                    initialise
WHILE i <= 5 DO              test
    PRINT i
    SET i = i + 1            update
```

Lose one of the three and…

no *initialise* → you count from garbage;

no *test* → the loop never stops;

no *update* → the loop never stops.

Today the same three parts appear in three different shapes: `while`, `do–while` and `for`.

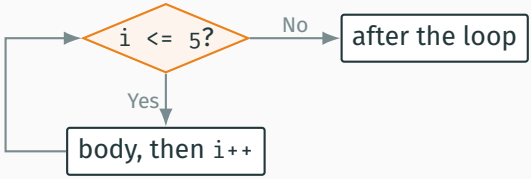
`while`: test first, then run

```
int i = 1;              // initialise

while (i <= 5) {        // test
    cout << i << " ";
    i = i + 1;          // update
}
cout << "| i = " << i;
```

Output

```
1 2 3 4 5 | i = 6
```



The test runs before every pass — including the first, so the body may run **zero** times.

The accumulator, now in C++

```cpp
int n;
cin >> n;

int total = 0;    // sum of nothing
int i = 1;

while (i <= n) {
    total += i;
    i++;
}
cout << "Sum = " << total << endl;
```

Pass	i	total
before	1	0
1	2	1
2	3	3
3	4	6
4	5	10

With `n = 4` it prints `Sum = 10`; with `n = 0` the body never runs and it prints `Sum = 0`. ✓

Two ways to loop forever

No update

```
int i = 1;
while (i <= 5) {
    cout << i << " ";
}   // i never changes
```

Prints `1` forever.

A test that is never exactly met

```
int i = 0;
while (i != 10) {
    cout << i << " ";
    i += 3;         // 0 3 6 9 12 ...
}
```

`i` jumps over 10. Use `<` or `<=`, not `!=`, when stepping by more than one.

And one more

`while (i <= 5);` — a semicolon after the condition makes the *empty statement* the body. The loop spins with nothing to change `i`.

Looping until a signal value

```
int x, count = 0, total = 0;

cout << "Enter marks, -1 to stop: ";
cin >> x;

while (x != -1) {
    total += x;
    count++;
    cin >> x;        // read the next one
}

cout << count << " marks, total " << total;
```

Input `80 95 70 -1`

```
3 marks, total 245
```

The shape to memorise

Read *once* before the loop, and read again at the *end* of the body. Forget the second read and the loop never ends.

break and continue

```
int i = 0;

while (i < 10) {
    i++;
    if (i % 2 == 0) continue;  // skip evens
    if (i > 7)      break;     // stop early
    cout << i << " ";
}
cout << "| end i = " << i;
```

break and continue

```
int i = 0;

while (i < 10) {
    i++;
    if (i % 2 == 0) continue;  // skip evens
    if (i > 7)      break;     // stop early
    cout << i << " ";
}
cout << "| end i = " << i;
```

Output

```
1 3 5 7 | end i = 9
```

`continue` jumps to the next test; `break` leaves the loop entirely.

Use them sparingly

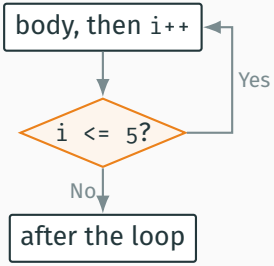
Both hide the real exit condition from the reader — and in a `while` loop, a `continue` that skips the update is one more way to loop forever.

do-while

do–while: run first, ask afterwards

```
int i = 1;

do {
    cout << i << " ";
    i++;
} while (i <= 5);
```



Output

```
1 2 3 4 5
```

Two details that cost marks

The test is at the bottom, so the body always runs **at least once**. And the closing `while (...)` needs a semicolon — this is the one loop that ends with one.

The difference shows up at zero

`while` — with `n = 0`

```
int i = 1;
while (i <= n) {
    cout << i << " ";
    i++;
}
```

Prints nothing. The test fails immediately.

`do-while` — with `n = 0`

```
int i = 1;
do {
    cout << i << " ";
    i++;
} while (i <= n);
```

Prints `1`. The body ran before anyone asked.

So choose by the question you are answering

"Maybe zero times" → `while`. "At least once, then decide" → `do-while`.

Where `do–while` is the right tool

```cpp
int age;

do {
    cout << "Enter age (0-120): ";
    cin >> age;
} while (age < 0 || age > 120);

cout << "Thank you" << endl;
```

Why it fits

You must ask *once* before you can possibly know whether the answer is valid.

Written with `while`, the same logic needs the prompt and the read duplicated — once before the loop, once inside.

The other classic use: a menu that reappears until the user chooses *Exit*.

for

`for`: the three parts, on one line

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
```

Part	In the header	When it runs
initialise	`int i = 1`	once, before anything else
test	`i <= 5`	before every pass — so zero passes is possible
update	`i++`	after every pass, then back to the test

The same three parts as the `while` loop — collected in one place, where you cannot forget one.

`for` and `while` are the same loop

`while`

```
int total = 0;
int i = 1;
while (i <= n) {
    total += i;
    i++;
}
```

`for`

```
int total = 0;
for (int i = 1; i <= n; i++) {
    total += i;
}
```

The rule of thumb

Use `for` when you know how many passes there will be — counting, stepping, walking over positions. Use `while` when the end depends on the data — a sentinel, a user answer, a condition that may never hold.

The `i` declared in the header exists *only* inside the loop.

Four `for` loops worth knowing

```cpp
// count down
for (int i = 5; i >= 1; i--)
    cout << i << " ";        // 5 4 3 2 1

// step by two
for (int i = 0; i <= 10; i += 2)
    cout << i << " ";        // 0 2 4 6 8 10
```

```cpp
// walk the alphabet
for (char c = 'a'; c <= 'e'; c++)
    cout << c;               // abcde

// accumulate
int f = 1;
for (int i = 1; i <= 5; i++)
    f *= i;                  // f = 120
```

The counter does not have to be an `int`, and it does not have to go up by one — but it must *move towards* the test.

Three `for` traps

1. The stray semicolon

```
for (int i = 1; i <= 5; i++);
    cout << i << " ";
```

The loop body is empty; the `cout` runs once — and will not even compile here, because `i` is already out of scope.

2. Off by one

```
for (int i = 1; i < 5; i++)   // 4 passes
for (int i = 1; i <= 5; i++)  // 5 passes
```

3. Changing the counter inside

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
    i++;              // now it skips
}                     // prints 1 3 5
```

And never count with a `double`

`for (double x = 0; x != 1.0; x += 0.1)` never stops — 0.1 is not exact in binary.

Nested loops: a loop inside a loop

```
for (int r = 1; r <= 4; r++) {
    for (int c = 1; c <= 4; c++) {
        cout << setw(4) << r * c;
    }
    cout << endl;      // end of the row
}
```

Output

```
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
```

How to read it

The inner loop runs completely for every single pass of the outer one: $4 \times 4 = 16$ passes in total. The `endl` belongs to the outer loop — that is what ends the row.

Your turn — predict the output

```
int count = 0;

for (int i = 1; i <= 20; i += 3) {
    if (i % 2 == 0) continue;
    count++;
    cout << i << " ";
}
cout << "| count = " << count;
```

Your turn — predict the output

```
int count = 0;

for (int i = 1; i <= 20; i += 3) {
    if (i % 2 == 0) continue;
    count++;
    cout << i << " ";
}
cout << "| count = " << count;
```

Output

```
1 7 13 19 | count = 4
```

`i` takes 1, 4, 7, 10, 13, 16, 19 — and `continue` drops the even ones.

Which loop should I use?

Loop	Test is	Use it when
`while`	before the body	the number of passes depends on the data; zero passes is legal
`do-while`	after the body	the body must run at least once — input validation, menus
`for`	before the body	you know how many passes: counting, stepping, tables

All three are interchangeable

Any loop can be written with any of them. Choose the one that makes the *reader* see the exit condition fastest.

Wrap-up

What to take away

Choosing

- `?:` chooses a *value*; `if`–`else` chooses an *action*. Both arms need one common type.
- `switch` compares one integral value against constants — and runs on through every label until a `break`.
- Ranges and combined conditions stay in `else if`.

Repeating

- Initialise, test, update — all three, every time.
- `while` may run zero times; `do`–`while` always runs once and ends with a semicolon.
- `for` puts the three parts where you can see them — use it when the count is known.
- In a nested loop the inner one runs completely for each outer pass.

Questions?